

MASTER IN
FULL STACK DEVELOPMENT E AI

TONGUE-API



DI LORENZO SOTTILE

[GITHUB](#)



OBIETTIVI

REALIZZARE UN'API RESTFUL CHE PERMETTA DI:

- 01 COSTRUIRE API REST SICURE E BEN STRUTTURATE
- 02 IMPLEMENTARE CRUD PER UTENTI, POST E INTERAZIONI
- 03 GESTIRE AGGREGATI E FILTRI AVANZATI
- 04 USARE MYSQL CON PREPARED STATEMENTS
- 05 INCLUDERE MIGRATIONS.SQL
- 06 PROTEGGERE DA SQL INJECTION
- 07 IMPLEMENTARE TEST AUTOMATICI (UNIT + E2E)

STACK TECNOLOGICO & DATABASE

- Node.js
- Express.js
- MySQL (mysql2)
- MVC Architecture
- dotenv
- Mocha + Sinon (unit test)
- Bash + cURL (E2E test)

3 **tabelle** principali:

- **users** (nickname, age, city)
- **posts** (title, created_at)
- **interactions** (type, interaction_time, user_id, post_id)

Relazioni semplici e chiare per aggregare le interazioni.

FUNZIONALITÀ PRINCIPALI

GESTIONE UTENTI
(USER MANAGEMENT)

GESTIONE POST
(POST MANAGEMENT)

GESTIONE INTERAZIONI
(INTERACTION MANAGEMENT)

ANALYTICS & AGGREGATI

FILTRI AVANZATI

SICUREZZA DELLE QUERY

TESTING COMPLETO
(UNIT + END-TO-END)

STRUTTURA SCALABILE
(MVC ARCHITECTURE)



ENDPOINTS CHIAVE

/API/USERS — CRUD UTENTI

/API/POSTS — CRUD + FILTRI

/API/POSTS/AGGREGATED — ANALYTICS

/API/INTERACTIONS — CRUD

/API/INTERACTIONS/FILTER — FILTRI AVANZATI



SICUREZZA & TESTING

SICUREZZA

- Tutte le query usano **prepared statements**
- Nessuna concatenazione SQL
- Nessuna vulnerabilità da SQL Injection
- Input sanitizzato

TESTING

Unit Test (Mocha + Sinon):

- Testano models/controller isolati
- DB simulato con stubs

End-to-End Test (test_api.sh):

- Crea user → post → interaction
- Controlla status code
- Controlla aggregati
- Output con OK/FAILED

THANK YOU!

LINKEDIN:

[HTTPS://WWW.LINKEDIN.COM/IN/LORENZO-SOTTILE/](https://www.linkedin.com/in/lorenzo-sottile/)

GITHUB REPO:

[HTTPS://GITHUB.COM/SADSOTTI/TONGUE-API](https://github.com/sadsotti/tongue-api)

GITHUB PROFILE:

[HTTPS://GITHUB.COM/SADSOTTI](https://github.com/sadsotti)

